

GameServer Portfolio

IOCP기반 MMORPG 서버

김성준

Content

1. Server Architecture

2. Architecture 설계 의도

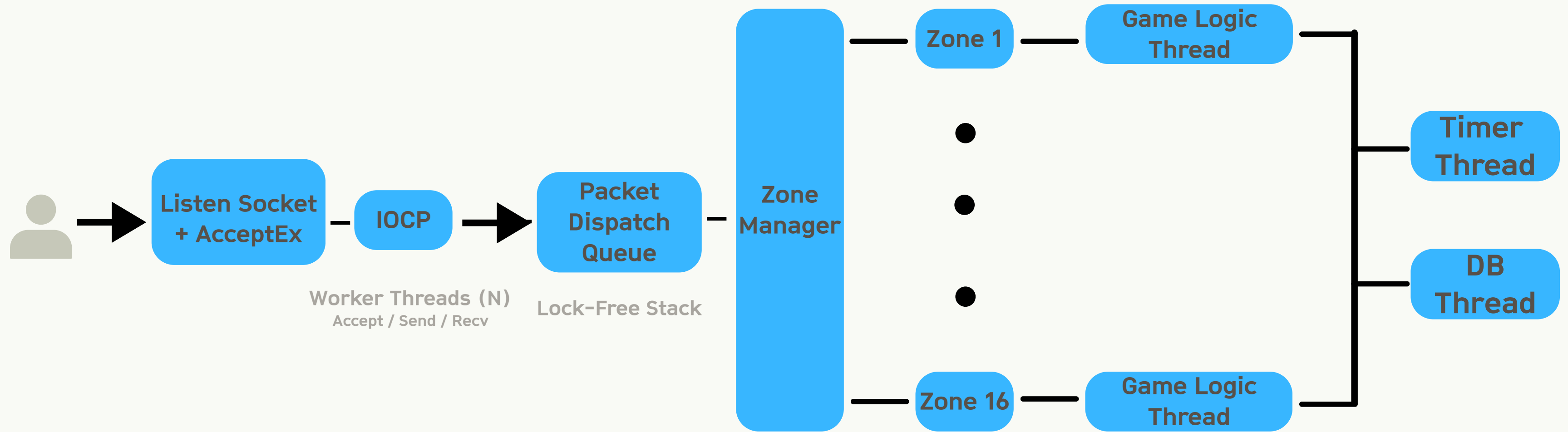
3. 성능 개선

4. StressTest 스펙

5. StressTest 시연

6. StressTest 결과

Server Architecture



Architecture 설계 의도

1. Multiple Producer & Single Consumer

WorkerThread, TimerThread, DBThread가 각각 패킷, 타이머 만료, DB결과를 이벤트로 생성 실제 게임 변경은 GameLogicThread 한 곳에서만 수행

2. 명확한 책임 분리

I/O, 타이머, DB는 병렬 처리, 게임로직만 단일 흐름으로 직렬화

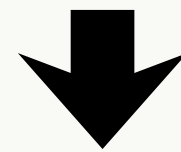
2. 정합성 우선 설계

플레이어 이동, 전투, AI, 오브젝트 상태 변경의 순서를 보장

성능 개선

기존 문제점

네트워크 I/O는 호출비용이 비쌈. 즉 자주 호출할 수록 서버 성능에 영향을 미침.
하지만 그렇다고 호출을 안할 순 없음 . 호출을 최소화 할 순 없을까 ?



Batch 처리 기법 도입 데이터 생성 또는 요청 시 실시간 처리 X
Queue 에 쌓아두었다가 일괄적으로 처리

Batch 를 위해 일부러 기다리지 않기 때문에 유저 지연 체감도 없음.
GameSession, TimerThread, GameLogicThread 적용

성능 개선

기존 문제점

DB 처리를 GameLogicThread 에서 처리해서 게임로직처리를 못하는 문제가 있었음.
DB 처리는 쿼리 결과를 기다리는 외부 I/O 작업 즉 CPU 를 사용하지 않는 작업임.
따라서 CPU 를 많이 사용하는 GameLogic 과 혼용하는 것이 문제

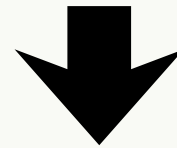


GameLogicThread 에서는 일감을 DBThread 에게 던지기만 하고, 콜백을 통해 결과만 처리하여 CPU 이용률을 효율적으로 개선

성능 개선

기존 문제점

GameLogicThread가 단 한개이기 때문에 동접이 늘어날 수록 병목현상이 심해짐.



Sector 상위 개념인 Zone을 도입. Zone마다 GameLogicThread를 들고 있게하여 Zone마다 게임로직을 병렬로 처리.

StressTest 스펙

봇 행동 패턴

행동	주기	내용
이동	400~600ms 랜덤	8방향(상·하·좌·우·대각) 중 이동 가능한 방향 랜덤 선택
일반 공격	400~600ms 랜덤	시야 내 몬스터 존재 시 공격 패킷 전송, facing 방향 반영
스킬 공격	5,200~7,000ms 랜덤	시야 내 몬스터 존재 시 스킬 패킷 전송
채팅	10,000~20,000ms 랜덤	7종 메시지 중 랜덤 전송 ("hello", "what's your name" 등)

부하 제어 알고리즘 (3단계 자동 조절)

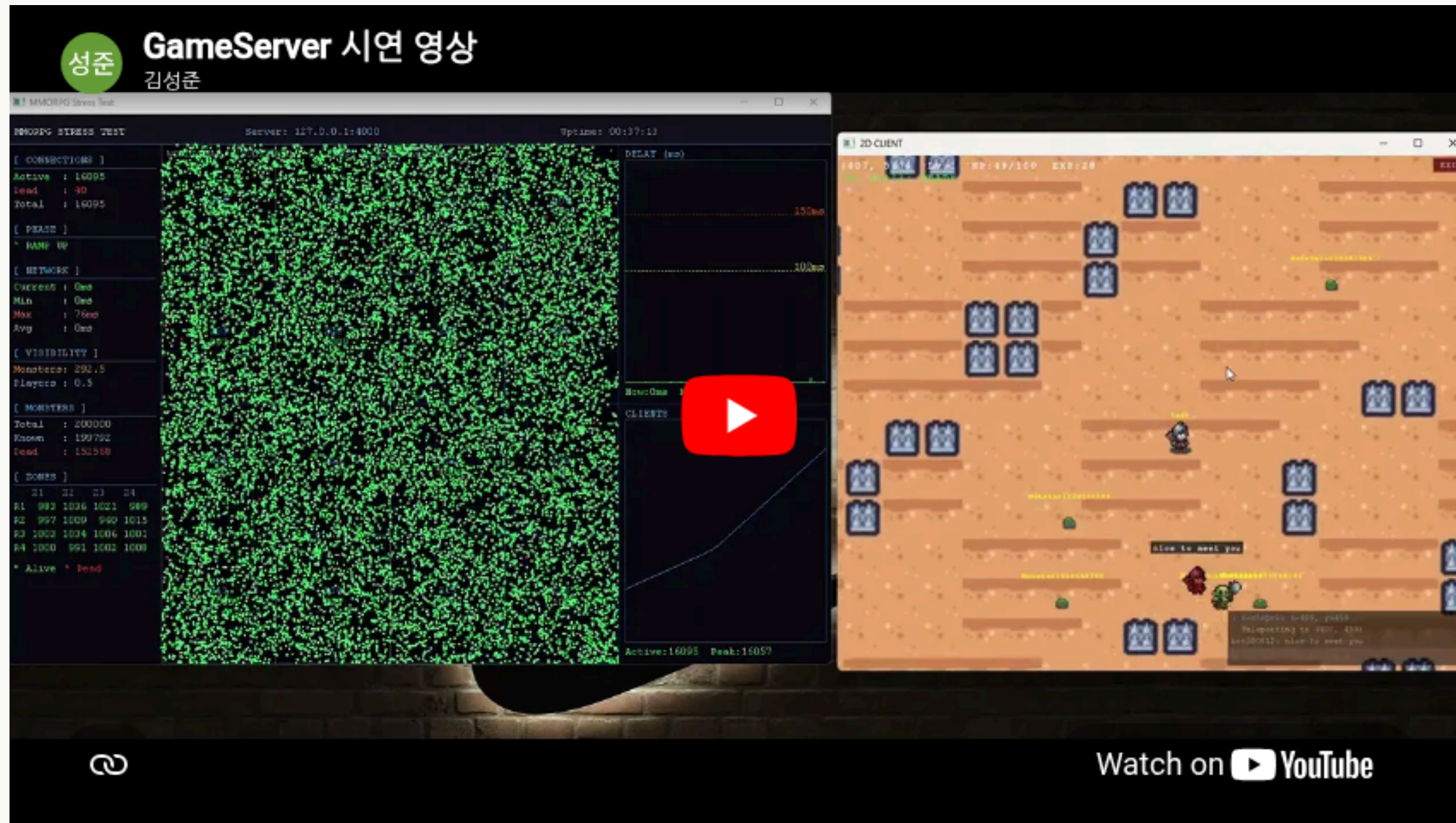
RAMP_UP → 접속 속도 50ms 간격으로 봇 추가

STABLE → 지연 100ms 초과 시, 접속 속도 500ms로 감속

REDUCING → 지연 150ms 초과 시, 봇 1개씩 강제 종료

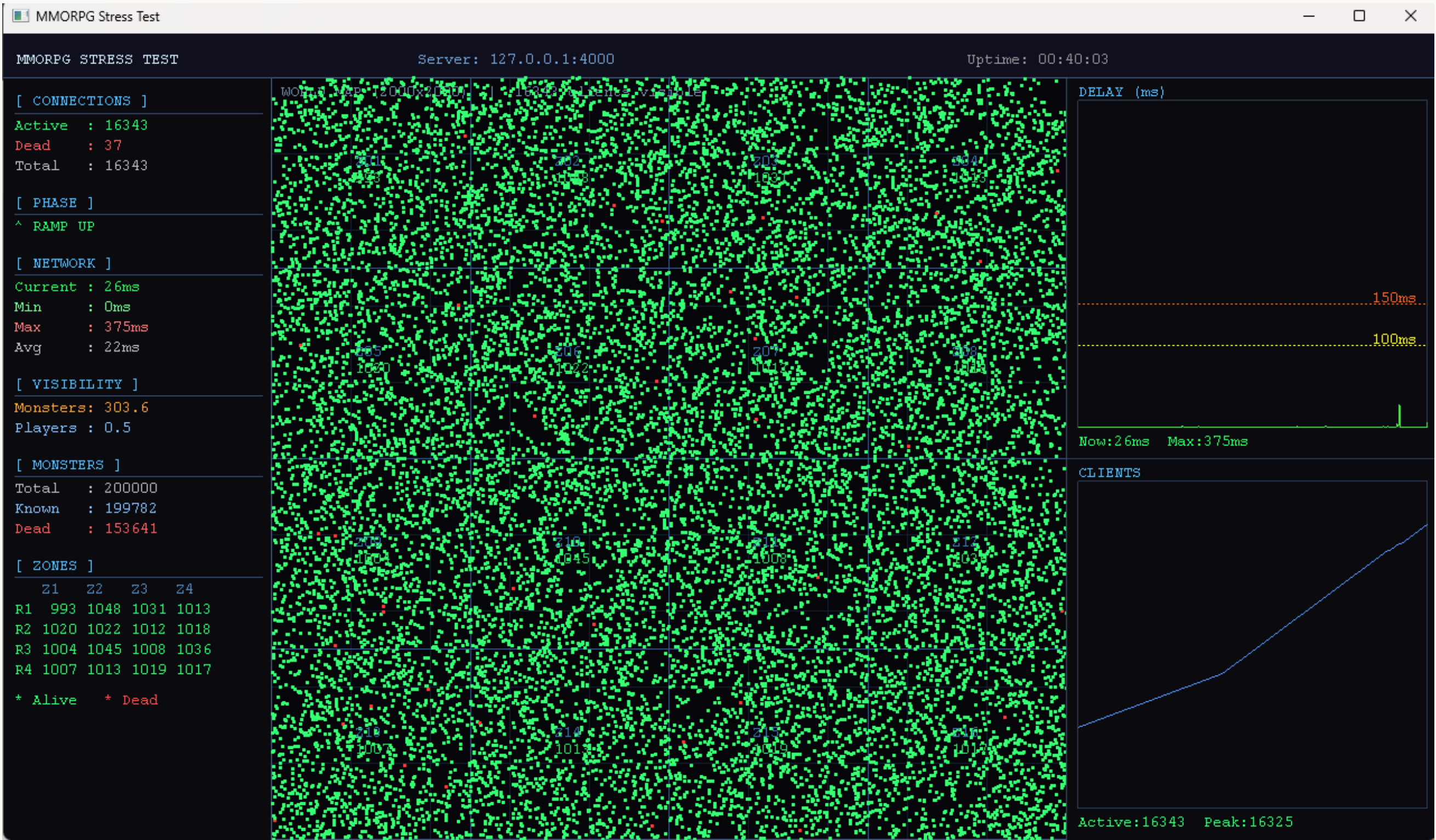
- 서버가 회복돼 안정 상태(STABLE 100회 연속)가 되면 → 다시 RAMP_UP 재개
- 최고 부하 도달 시 maxLimit을 기록, 그 95% 이내로 수렴

Stress Test 시연



<https://www.youtube.com/watch?v=GGiP0ix0Mgw>

Stress Test 결과



단락 텍스트